Mini-Lakehouse: Detailed Design Document

Nov 4, 2026 · @Naul

1. Overview

Mini-Lakehouse is a small, production-shaped data platform: game telemetry flows through Kafka into Apache Iceberg tables on object storage and is queried with an OLAP engine. It is built to show platform-building skill (storage, streaming, compute, and an interface for internal customers), not pipeline-running skill.

Goals

- Demonstrate event-driven design with Kafka (Redpanda locally): keys, partitions, consumer groups, replay.
- Demonstrate lakehouse design with Iceberg: partitioning, schema evolution, compaction, time travel.
- Show sound architecture in Python: ports and adapters, typed contracts, tested domain core.
- Treat other engineers as internal customers: a documented ingestion contract, a self-serve table catalog, and a query path.
- Run it all in containers, with a Kubernetes (kind) deployment as a stretch.

Non-goals

- No multi-node cluster tuning, no real auth/SSO, no petabyte benchmarks.
- No ML or dashboards; one demo notebook is enough.
- No claim of production experience with Kafka or Iceberg. The README says this is a learning build with production-style design.

Success criteria

- One command (`make up`) starts the full stack and a demo producer.
- Killing the consumer mid-stream and restarting it yields zero duplicate and zero lost rows in the target table (verified by an automated test).
- A schema change (new optional field) lands without rewriting data or breaking existing queries.
- Query latency and ingest throughput are measured and published in the README, with the hardware stated.

Fit to the target role (NestAI Data Platform Chapter)

Posting requirement	Where this project shows it
Python and software architecture	Hexagonal core, typed ports, pytest suite
Event-driven streaming (Kafka)	Section 5: topics, partitioning, delivery semantics
OLTP and OLAP	Postgres for the control-plane catalog (OLTP), DuckDB/Trino over Iceberg (OLAP)
Lakehouse, Iceberg	Section 6: commit protocol, compaction, snapshots
Distributed systems reliability	Section 8: failure modes, idempotency, replay
Kubernetes, Docker (nice to have)	Section 9: compose plus kind manifests
Go or Rust (nice to have)	Optional Go compaction/maintenance job (stretch)

2. Requirements

The build is scoped to what one engineer can finish and test in two weeks. Must-haves come first; stretch items are cut without hurting the story.

Functional requirements

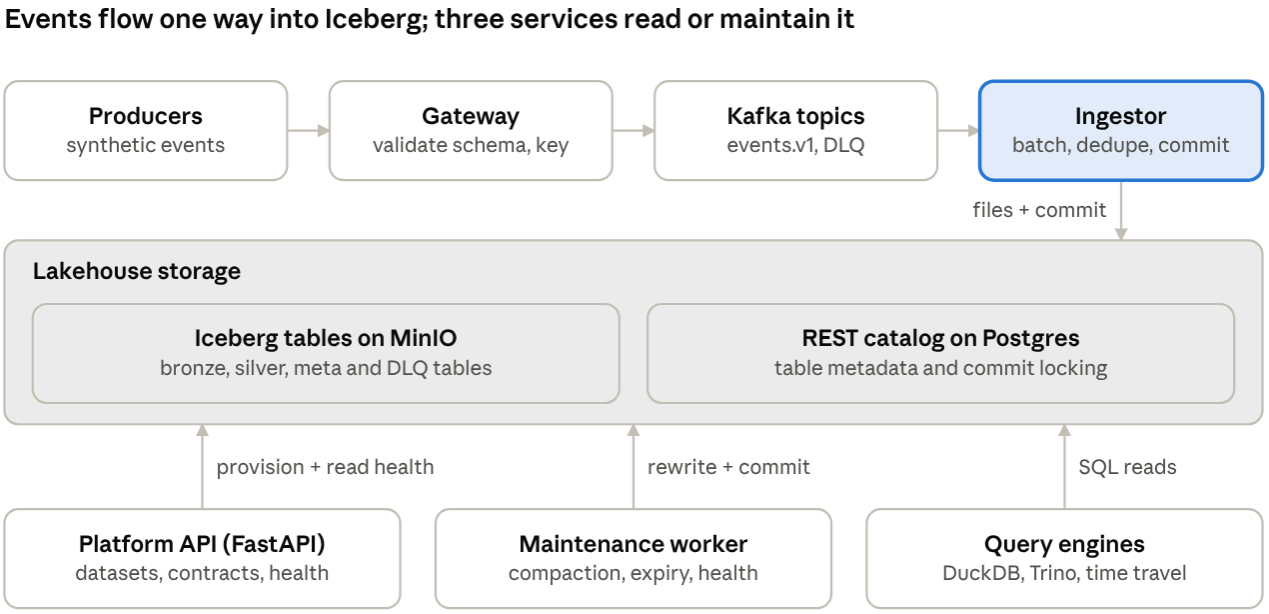
ID	Requirement	Priority
F1	Producer emits synthetic game telemetry (sessions, matches, purchases) at a configurable rate, with realistic skew and late events	Must
F2	Events are validated against a versioned schema before entering Kafka (schema registry or in-repo JSON Schema)	Must
F3	Consumer writes events to Iceberg tables with at-least-once delivery and idempotent writes (effectively once)	Must
F4	Invalid events go to a dead-letter topic with the failure reason	Must
F5	Tables are queryable with SQL (DuckDB; Trino optional) including time travel to an earlier snapshot	Must

ID	Requirement	Priority
F6	Schema evolution (add optional column) without rewriting existing data	Must
F7	Maintenance job: small-file compaction and snapshot expiry on a schedule	Must
F8	Platform API (FastAPI): register a dataset, view schema and lineage, check table health	Should
F9	Late-data handling: events up to 24 h late land in the correct partition	Should
F10	Go (or Rust) maintenance worker as an alternative to the Python one	Could

Non-functional requirements

Area	Target
Throughput	5,000 events/s sustained on a laptop (8 cores, 16 GB), measured and published
Ingest-to-query latency	Under 60 s from produce to queryable in Iceberg
Durability	No loss on consumer or broker restart; Kafka replication factor 1 locally, 3 documented for prod
Correctness	Zero duplicates after forced consumer crash, checked by test
Operability	Metrics, structured logs with correlation IDs, health endpoints
Portability	`docker compose up` on Linux and macOS; no cloud account needed
Code quality	Typed Python (mypy strict on the core), 80%+ coverage on domain code, CI green

3. Architecture



architecture · 8 components, 3 layers

Telemetry is validated at the gateway, buffered in Kafka, written to Iceberg by the ingestor, and read, maintained, or provisioned by three separate services that share only the catalog and the tables.

Technology choices

Layer	Choice	Why	Production alternative
Broker	Redpanda	Single binary, Kafka API, fast local start	Apache Kafka (same client code)
Object storage	MinIO	S3 API on a laptop	S3 or GCS
Table format	Apache Iceberg v2	Open format, hidden partitioning, snapshots, schema evolution by column ID	Same
Catalog	Iceberg REST catalog on Postgres	Engine-neutral; one source of truth for DuckDB and Trino	Managed catalog (Polaris, Nessie, Glue)
Ingest runtime	Python consumer with PyIceberg	Matches the role's Python requirement; keeps the commit protocol visible	Flink or Spark Structured Streaming

Layer	Choice	Why	Production alternative
Query	DuckDB, optionally Trino	Zero-ops OLAP plus a distributed engine for comparison	Trino or Spark cluster
Platform API	FastAPI with Postgres	Typed, self-documenting API for internal customers	Same
Packaging	Docker Compose, then kind	Reproducible locally; cloud-native shape	Managed Kubernetes

Architectural principles

- **Hexagonal core.** Domain logic (dedupe, partition rules, contract checks) has no I/O and is tested without containers. Kafka, Iceberg, and Postgres sit behind ports.
- **Independent processes.** Ingestion, maintenance, and the API deploy and scale separately; a maintenance failure never blocks ingestion.
- **Contracts at the edges.** Schemas are versioned and checked on the way in; compatibility is checked on the way out.
- **Open formats only.** Everything stored is Parquet and Iceberg, so any engine can read it, which matches the sovereignty and interoperability theme in the posting.

4. Data model

One event envelope, three event types, and a small set of Iceberg tables. Raw events are kept as-is (bronze); a derived table serves analytics (silver).

Event envelope (schema v1)

Field	Type	Notes
event_id	UUID (v7)	Globally unique; the idempotency key
event_type	string	`session_start`, `match_end`, `purchase`
event_time	timestamp (UTC)	When it happened on the client
ingest_time	timestamp (UTC)	Set by the gateway; used for late-data metrics
player_id	string	Kafka message key, so one player's events stay ordered
schema_version	int	Drives validation and evolution

Field	Type	Notes
payload	struct	Type-specific fields (match duration, item_id, price_cents, etc.)

Iceberg tables

Table	Layer	Partitioning	Sort order	Purpose
`bronze.events`	Raw	`day(event_time)`, `bucket(16, player_id)`	`event_time`	Every valid event, append-only
`bronze.events_dlq`	Raw	`day(ingest_time)`	none	Rejected events plus error reason
`silver.player_daily`	Derived	`month(day)`	`player_id`	Per-player daily aggregates (sessions, matches, spend)
`meta.table_health`	Control	none	none	File counts, sizes, last compaction (written by the maintenance job)

Design decisions

- **Partition by event time, not ingest time.** Queries filter on when things happened; late events land in old partitions and compaction fixes the small files that creates.
- **Bucket on player_id (16 buckets)** spreads writes and speeds up per-player lookups without creating millions of partitions.
- **Idempotency via `event_id`.** The consumer writes with a MERGE/dedupe step keyed on `event_id` per micro-batch, and the commit records the Kafka offsets in the Iceberg snapshot summary so a restart resumes from exactly the committed position.
- **Schema evolution.** v2 adds optional `payload.platform` (string). Iceberg adds the column by ID with no data rewrite; old rows read as null. The producer and validator accept both versions during a transition window.

- **Naming.** Namespaces `bronze`, `silver`, `meta`; tables are lowercase snake_case; every table has an owner and description property set in the catalog.

5. Streaming layer

Redpanda (Kafka API compatible, single binary) runs locally; the code only uses the Kafka protocol, so it moves to Apache Kafka unchanged.

Topics

Topic	Partitions	Key	Retention	Purpose
`telemetry.events.v1`	12	`player_id`	7 days	Validated events from the gateway
`telemetry.events.dlq`	3	`event_id`	30 days	Events that failed validation or write
`platform.commits`	1	table name	7 days	Commit notifications so downstream jobs can react (stretch)

Producer (gateway)

- Validates each event against the JSON Schema for its `schema_version`; invalid events are sent to the DLQ with a reason code.
- Settings: `acks=all`, `enable.idempotence=true`, `linger.ms=20`, `compression=zstd`. These trade a little latency for throughput and safe retries.
- Keys by `player_id`, so all of a player's events share a partition and keep their order.

Consumer (ingestor)

- One consumer group, `lakehouse-ingestor`; scaling out means adding instances up to the partition count (12).
- Auto-commit is **off**. The loop is: poll a micro-batch (up to 10,000 events or 10 s), dedupe by `event_id`, write Parquet files, commit the Iceberg snapshot with the batch's end offsets in the snapshot summary, then commit offsets to Kafka.
- This ordering gives **at-least-once delivery with idempotent output**. If the process dies between the Iceberg commit and the Kafka offset commit, the restart reads the offsets stored in the latest snapshot, skips what is already written, and continues. The result is effectively-once in the table.
- Backpressure: the poll loop pauses partitions when the write queue passes a high-water mark, instead of buffering unbounded memory.

- Rebalances: on partition revoke, the consumer flushes and commits the in-flight batch before releasing.

Why not Kafka transactions or Flink? Iceberg commits are not part of a Kafka transaction, so exactly-once across both needs the offsets-in-snapshot approach anyway. Flink would be the production choice at scale; for this build a plain Python consumer keeps the moving parts visible and the design explainable in an interview.

6. Lakehouse layer

Storage and catalog

- **Object storage:** MinIO (S3-compatible) locally. The same code points at S3 or GCS by changing the endpoint and credentials.
- **Table format:** Apache Iceberg v2, Parquet data files with zstd compression, target file size 128 MB.
- **Catalog:** Iceberg REST catalog (Lakekeeper or the Tabular-compatible reference image) backed by Postgres. Postgres here is the OLTP side: small, transactional metadata. The tables themselves are the OLAP side.
- **Library:** PyIceberg for writes and maintenance from Python.

Commit protocol

1. Writer buffers a micro-batch and writes Parquet files to a staging path.
2. Writer calls an Iceberg `append` (or `overwrite` for the dedupe path) and commits atomically through the catalog.
3. On a commit conflict (another writer touched the table), the writer refreshes metadata and retries with exponential backoff, up to 5 attempts, then routes the batch to the DLQ and alerts.
4. The snapshot summary carries `kafka.offsets` (per partition) and `ingestor.batch_id` for exactly-once recovery.

A single writer per partition range is the default. Concurrent writers are supported through optimistic concurrency, and a test forces a conflict to prove the retry path.

Maintenance (the part most pipelines skip)

Job	Schedule	What it does	Why
Compaction	Every 15 min on hot partitions, nightly on all	Rewrites small files into ~128 MB files	Streaming creates many small files that slow queries

Job	Schedule	What it does	Why
Snapshot expiry	Daily	Expires snapshots older than 7 days, keeps the last 20	Bounds metadata size, keeps time travel useful
Orphan file cleanup	Weekly	Deletes unreferenced files older than 3 days	Reclaims space after failed commits
Health report	Every 15 min	Writes file counts, average size, and partition skew to `meta.table_health`	Gives platform users visibility without asking us

The maintenance worker is a separate process with its own container, so it can be scaled, scheduled, or rewritten (Go is the stretch option) without touching ingestion.

Key trade-offs to document in the README

- Micro-batch interval (10 s) trades freshness against file count; a shorter interval means more small files and more compaction work.
- Copy-on-write dedupe is simple and fast to read but rewrites files; merge-on-read would reduce write cost and is noted as a future option.
- Hidden partitioning (`day(event_time)`) keeps query authors from needing to know the physical layout.

7. Query layer and platform API

This is the part that treats other engineering teams as customers: they should discover data, trust its shape, and query it without knowing how ingestion works.

Query engines

- **DuckDB** (default): reads Iceberg directly through its Iceberg extension; zero infrastructure and used in the demo notebook and benchmarks.
- **Trino** (optional): one container with the Iceberg connector, to show a multi-user distributed engine reading the same tables and to compare results.
- Both read through the REST catalog, so there is a single source of truth for table metadata.

Platform API (FastAPI, Postgres-backed)

Endpoint	Purpose
`POST /datasets`	Register a dataset: name, owner, schema, retention, PII flag. Creates the topic, DLQ, and Iceberg table from one declaration
`GET /datasets/{name}`	Schema, owner, partition spec, current snapshot, freshness
`GET /datasets/{name}/lineage`	Upstream topic and downstream derived tables (stored as simple edges)
`GET /datasets/{name}/health`	Small-file ratio, last compaction, consumer lag, DLQ rate
`POST /schemas/{name}/versions`	Propose a new schema version; the API rejects breaking changes (removed or retyped fields)
`GET /healthz`, `GET /metrics`	Liveness and Prometheus metrics

Data contracts

- A dataset is declared in a small YAML file (name, owner, schema, key, retention). CI validates it and the platform provisions everything from it.
- Compatibility rules: adding optional fields is allowed; removing fields, renaming, or changing types is rejected unless a new major version is created.
- Contracts live in the repo under `contracts/`, so a change is a reviewed pull request.

Why a declarative contract? At NestAI scale, many chapters will onboard datasets. Provisioning by declaration keeps that repeatable and reviewable, and it is the main demonstration of platform thinking in this project.

8. Reliability and observability

Failure modes and responses

Failure	Effect	Response	Test
Ingestor crashes mid-batch	Batch partly written	Restart resumes from offsets stored in the last snapshot; orphan files are cleaned up later	Kill -9 during load, then compare row counts and distinct `event_id`

Failure	Effect	Response	Test
Crash between Iceberg commit and Kafka offset commit	Batch replayed	Dedupe by `event_id` plus snapshot offsets skip it	Inject a crash at that exact point
Broker restart	Producer and consumer reconnect	Idempotent producer retries; consumer resumes from its group offset	Restart broker under load
Iceberg commit conflict	Commit rejected	Refresh metadata, retry with backoff, then DLQ and alert	Two writers on one table
Object storage unavailable	Writes fail	Pause consumption, retry, expose lag metric; no data dropped because Kafka retains it	Stop MinIO for 60 s
Poison message	Validation or parse error	Route to DLQ with reason code; the pipeline keeps going	Send malformed JSON
Schema change breaks consumers	Readers fail	Contract check rejects breaking changes before deploy	CI test with a breaking contract
Small-file explosion	Slow queries	Compaction, plus an alert when average file size drops below 16 MB	Run a burst, then watch health metrics

Observability

- **Metrics (Prometheus format):** consumer lag per partition, batch size, commit latency, events/s, DLQ rate, commit retries, small-file ratio, snapshot age.
- **Logs:** structured JSON with `batch_id`, `partition`, `offset_range`, and a correlation ID carried from producer to table, reusing the approach from your Agent Platform work.
- **Dashboards:** one Grafana dashboard (optional) with lag, throughput, and table health; screenshots go in the README.
- **SLO example to state in the README:** 99% of events queryable within 60 s of ingest, measured by the `ingest_time` to commit-time delta.

Security basics (scoped honestly)

- Secrets come from environment variables or Kubernetes secrets, never from the repo.
- Non-root containers, pinned image versions, dependency scanning in CI.
- The README lists what is out of scope (authn/z, encryption at rest, multi-tenancy), since a defence-sector reviewer will look for that awareness.

9. Deployment and repository

Local stack (docker compose)

Service	Image / role	Port
redpanda	Kafka-compatible broker	9092
redpanda-console	Topic and consumer-group UI	8080
minio	S3-compatible object storage	9000 / 9001
postgres	Catalog and platform metadata (OLTP)	5432
iceberg-rest	Iceberg REST catalog	8181
gateway	Python producer/validator service	8001
ingestor	Python Kafka-to-Iceberg consumer	none
maintenance	Compaction, expiry, health report	none
platform-api	FastAPI dataset and health API	8000
trino (optional)	Distributed SQL engine	8082
prometheus, grafana (optional)	Metrics and dashboard	9090 / 3000

Kubernetes (stretch, kind)

- One Helm chart or Kustomize overlay for the four application services, with resource requests and limits, readiness and liveness probes, and a ConfigMap and Secret for settings.
- The ingestor runs as a Deployment scaled to 1 to 4 replicas; the README shows a rebalance as replicas are added.
- The maintenance worker runs as a CronJob.
- Broker, MinIO, and Postgres use community Helm charts; they are not rewritten by hand.

Repository layout

```
mini-lakehouse/
  contracts/            # dataset YAML contracts + JSON Schemas
  src/
    domain/             # pure logic: events, dedupe, partition rules (no
I/O)
    ports/              # interfaces: EventSource, TableWriter, Catalog
    adapters/           # kafka, iceberg, minio, postgres implementations
    gateway/            # producer + validation service
    ingestor/           # consume loop, batching, commit protocol
    maintenance/        # compaction, expiry, health
    platform_api/       # FastAPI app
  tests/
    unit/  integration/  chaos/
  deploy/
    compose/  k8s/
  docs/
    architecture.md  decisions/   # ADRs, one per major choice
  notebooks/demo.ipynb
  Makefile  pyproject.toml
```

CI (GitHub Actions)

- On every push: ruff, mypy (strict on `domain` and `ports`), unit tests, contract validation.
- On pull requests: integration tests with Testcontainers (Redpanda, MinIO, REST catalog), then a Docker build smoke test.
- On main: build and push images; chaos tests run nightly because they are slower.
- Write short ADRs (architecture decision records) for the big choices: Redpanda vs Kafka, offsets-in-snapshot vs transactions, copy-on-write vs merge-on-read. Reviewers read these first.

10. Testing and benchmarks

Test layers

Layer	Scope	Tools
Unit	Domain logic: dedupe, partition assignment, contract compatibility rules	pytest, hypothesis (property tests for dedupe and ordering)
Integration	Gateway to Redpanda to Iceberg on MinIO, real containers	pytest with Testcontainers

Layer	Scope	Tools
Chaos	Kill the ingestor, restart the broker, stop MinIO, force commit conflicts	pytest plus Docker control, run nightly
Contract	Compatible and breaking schema changes are accepted or rejected as designed	pytest in CI
Load	Throughput, latency, small-file growth	Python load generator, results saved as CSV

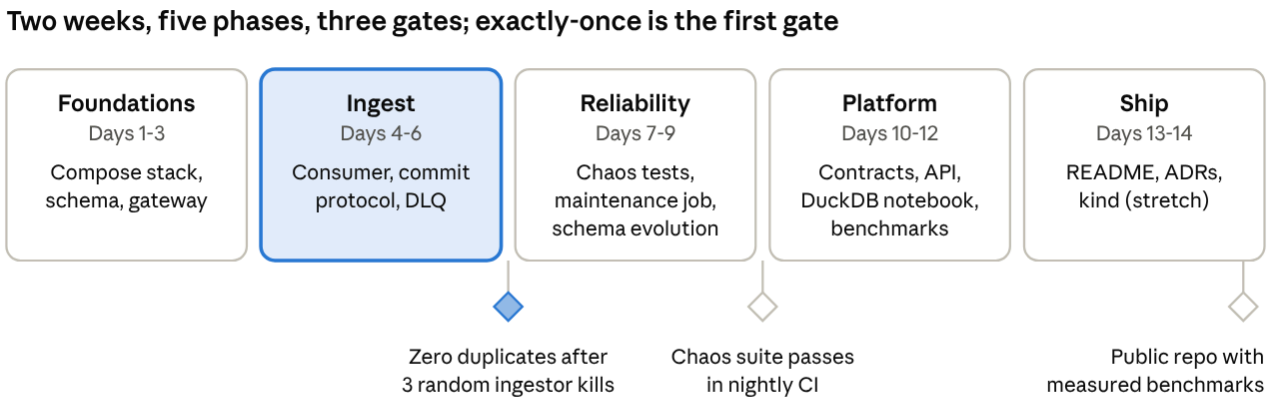
The test that matters most: produce 1,000,000 events with a known set of `event_id`s, kill the ingestor three times at random points, then assert the table holds exactly those 1,000,000 distinct IDs. This proves the exactly-once claim and goes in the README as the headline result.

Benchmarks to publish (with hardware stated)

Metric	Method	Target
Sustained ingest	Producer at 5,000 events/s for 10 min	Lag stays under 10 s
Ingest-to-query latency	`ingest_time` to snapshot commit time, p50 and p99	p99 under 60 s
Query speed	Daily active players for one day, with and without compaction	Compaction gives a measurable speedup; report the ratio
Small-file effect	File count and average size over a 30-minute run	Average file under 16 MB before compaction, about 128 MB after
Recovery time	Seconds from ingestor restart to first new commit	Under 30 s

Report measured numbers only. If a target is missed, state that and explain why; an honest miss with analysis reads better than a claimed pass.

11. Delivery plan, risks, and portfolio story



If behind: cut the Go worker, Trino, kind manifests, and Grafana, in that order.

delivery plan · 5 phases, 3 gates

The plan fits two weeks at about 4 to 6 focused hours a day. The first gate (zero duplicates after random kills) is the one to protect: it proves the core claim of the project.

Risks

Risk	Likelihood	Mitigation
PyIceberg write API is less mature for streaming than Spark or Flink	Medium	Keep the writer behind a port; use micro-batch appends; document the limit and name Flink as the scale-up path
Exactly-once is hard to prove, and easy to claim without proof	Medium	The kill-and-count test is a gate; no claim goes in the README without it passing
Scope creep (Trino, Kubernetes, Go worker, Grafana)	High	Fixed cut line in the plan above; all four are optional
Laptop runs out of memory with the full stack	Medium	Compose profiles: `core` (broker, MinIO, catalog, ingestor) and `extras` (Trino, Grafana)
Small-file explosion hurts the demo	Low	Build compaction before benchmarking, and show the before and after numbers

Portfolio story (what the README should do)

1. Open with the problem: many teams need a shared, reliable data foundation, and the platform must be self-serve.

2. Show the architecture diagram and the three headline results: exactly-once under kills, ingest-to-query latency, and compaction speedup.
3. Link the ADRs so a reviewer sees the trade-offs, not only the code.
4. Add a short "what changes at scale" section: Flink for ingest, merge-on-read, multi-tenancy and authn/z, lineage with OpenLineage, replication factor 3, and tiered storage.
5. State plainly that this is a learning build with production-style design, not production experience.

CV and application use

- One CV line: "Built an event-driven lakehouse platform (Kafka, Iceberg, Python) with exactly-once ingestion verified by chaos tests; declarative dataset contracts; compaction and snapshot maintenance." Use it only once the gates pass.
- Link the repo and the architecture diagram in the cover letter, and name the two design decisions you are proudest of.
- Pair it with Agent Platform: that shows the same event-driven thinking on cloud-managed services, and this project shows it on open-source infrastructure.